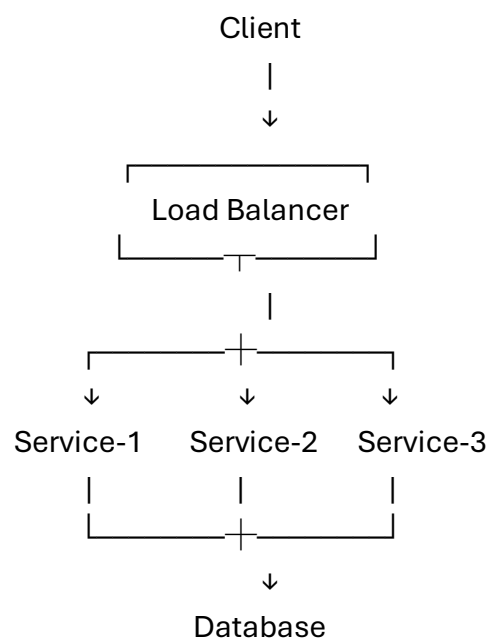


Task 02 — Load Balancing & Scalability

Objective

Implement a system where a **client** sends requests to a **Load Balancer**, which distributes the requests among multiple instances of a service.

Architecture



1. Load Balancing

A **Load Balancer** distributes incoming client requests across multiple service instances.

Example:

Request 1 → Service 1
Request 2 → Service 2
Request 3 → Service 3
Request 4 → Service 1

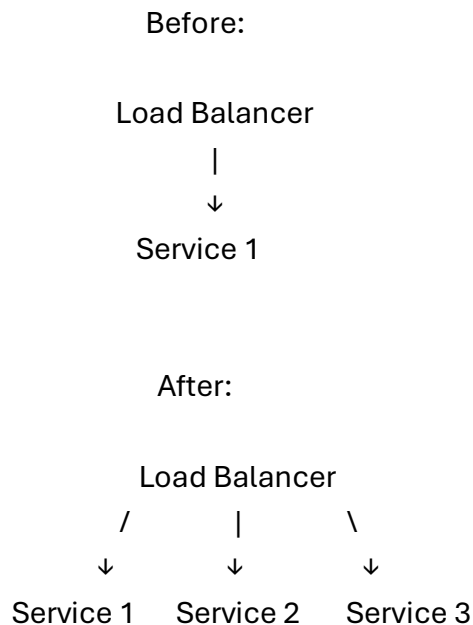
This prevents one service instance from receiving all the traffic.

Common load-balancing algorithms

- **Round Robin** – requests are distributed sequentially.
- **Least Connections** – request goes to the instance with the fewest active connections.
- **Weighted Round Robin** – more powerful servers receive more requests.

2. Horizontal Scaling

Horizontal scaling means **adding more service instances/servers**.



Advantage: The application can handle more users and requests.

3. Vertical Scaling

Vertical scaling means **increasing the resources of an existing server**, such as:

- More CPU
- More RAM
- More storage

Before:

Service 1
4 GB RAM
2 CPU

After:

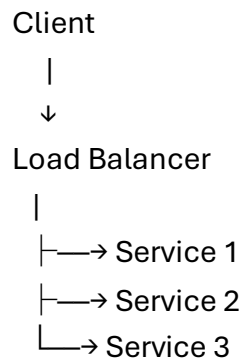
Service 1
16 GB RAM
8 CPU

Limitation: There is a physical/resource limit to how much one server can be upgraded.

4. Stateless Services

A **stateless service** does not store user session information inside a particular service instance.

For example:



A user's next request can go to **any service instance**.

For shared session/state information, we can use an external store such as **Redis** or a database.

5. Spring Boot Implementation

Create a simple Spring Boot service with multiple instances.

For example:

Service application:

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from Service Instance";
    }
}
```

Run the application on different ports:

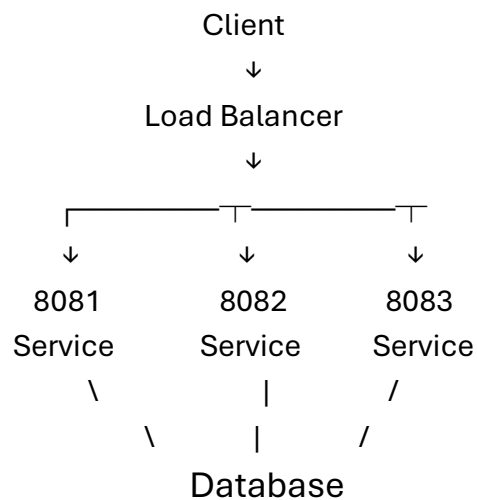
Instance 1 → Port 8081

Instance 2 → Port 8082

Instance 3 → Port 8083

The Load Balancer distributes requests between these instances.

Final Flow



Key Difference

Concept	Meaning
Load Balancing	Distributes requests across instances
Horizontal Scaling	Adds more instances
Vertical Scaling	Adds more CPU/RAM to an instance
Stateless Service	Does not depend on local server session/state

Task 02 is completed with a Load Balancer → Multiple Service Instances architecture, demonstrating horizontal scaling, vertical scaling, and stateless services.